软件课程设计报告(2026 年春季学期)

序号：

题目：MuseFlow AI 场景配乐工作台

系别：计算机科学与技术

班级：计 23-2 班

姓名：周凯君

学号：23101020222

总成绩：

评语：

指导教师签字：　　日期：

目录

1 课题主要内容介绍.....2
1.1 课题概述.....2
1.2 产品需求文档.....2
1.3 运行环境说明.....2
2 设计过程介绍（分功能模块）.....3
2.1 总体架构与技术选择.....3
2.2 数据关系与异常边界.....3
3 系统实现.....4
3.1 提示词模板与结构化设计稿.....4
3.2 受约束 Agent 工作流与任务调度.....4
3.3 ElevenLabs 音乐模型接入.....5
3.4 音频后处理与严格时长.....5
3.5 持久化、版本分支与对比.....6
3.6 前后端框架应用与桌面部署.....6
3.7 典型产品项目的全过程实现.....7
4 系统测试.....8
4.1 测试方法与覆盖范围.....8
4.2 ElevenLabs 真实调用结果.....8
4.3 问题修正与结果边界.....8
5 项目安装配置文档.....9
6 项目总结.....9
附录：全部源代码电子附件.....9

摘要

MuseFlow 是面向短视频、游戏及叙事场景的本地优先 AI 配乐工作台。系统以 C++20 实现业务与模型调用，以 Vue 3 构建交互界面，并由 Qt 6 提供独立桌面窗口。用户可把场景转换为结构化音乐设计稿，调整节奏、乐器、歌词和时间轴，再调用音乐模型生成音频，完成严格时长处理、版本保存与 A/B 对比。项目将模型输出视为待验证数据，通过 Schema、后台任务队列与不可变版本快照，提高生成过程的可控性和可追溯性。

关键词：C++；提示词模板；工作流式 Agent；Vue；音乐生成；SQLite

课程要求对应说明

正文覆盖“提示词模板使用”“Agent 应用开发”“前后端框架的应用部署”三个方向。Agent 采用模型生成设计稿、人工确认、应用调用固定工具链的受约束工作流；并非由大模型自主选择工具。项目没有使用 LangChain、向量数据库或 RAG，不将这些技术写成已实现功能。完整源码以报告末尾超链接附件提供，关键代码在正文中以可复制文本展示。

1 课题主要内容介绍

1.1 课题概述

传统场景配乐需要反复描述曲风、修改段落并管理试听文件，单次“输入提示词—下载音频”的方式难以保留创作过程。本课题设计 MuseFlow，把自然语言需求、可编辑设计稿、音乐生成任务和音频版本关联起来，形成“描述—设计—确认—生成—试听—分支改稿”的完整产品流程。

系统的目标不是训练新的音乐模型，而是实现稳定、可替换的模型接入和本地创作管理。文本服务负责把意图转成结构化参数；音乐服务负责音频生成；本机负责校验、时长处理和文件保存。即使没有 API Key，也可以用 Mock 模式演示完整流程。

1.2 产品需求文档

需求	输入及行为	验收标准
场景转设计稿	输入用途和场景；生成曲风、BPM、调性、乐器、歌词和段落	设计稿通过字段及时间轴校验后进入编辑区
设计稿编辑与改稿	直接编辑参数，或提交自然语言修改要求	非法输入明确提示；失败保留原稿；提示词与参数保持一致
多服务商接入	文本支持 Mock/DeepSeek；音乐支持 Mock/Lyria/SongAPI/ElevenLabs	分别保存服务商与 Key；连接测试可见；不静默切换 Mock
生成、播放与导出	30 秒试听或完整歌曲；WAV/MP3；可选严格时长	后台生成不阻塞界面；可播放、定位、下载并保留原始音频
项目与版本管理	项目保存、重命名、搜索；版本分支、评分、备注和 A/B 对比	重启可恢复；生成新版本不覆盖旧文件；父子关系有效
安全与可靠性	仅监听本机；校验远端响应；Key 会话管理；异常分类	日志和公开设置不回显 Key；收费生成失败不自动重发

1.3 运行环境说明

开发与验证环境为 Linux。后端采用 C++20、CMake（≥3.20）、libcurl、SQLite3、nlohmann/json 和 cpp-httplib；桌面采用 Qt（≥6.4）的 Widgets、Network、WebEngineWidgets；前端采用 Vue 3、TypeScript、Pinia、Vue Router 与 Vite。构建前端需要 Node.js ≥22.12 和 npm；音频处理需要 FFmpeg/ffprobe。

本机服务地址为 http://127.0.0.1:18080，开发前端地址为 http://127.0.0.1:5173。数据、设置和缓存遵循 XDG 目录规范。真实生成需要网络、对应账户权限及余额；当前实际验证的音乐模型是 ElevenLabs music_v1。

2 设计过程介绍（分功能模块）

2.1 总体架构与技术选择

采用“桌面外壳—前端界面—本地业务服务—模型适配器”的分层设计。C++ 后端统一处理模型鉴权、校验和存储，浏览器页面只访问本机 API。选择独立接口而非在页面直接调用服务商，可以复用错误处理和测试替身，也便于新增音乐模型而保持创作台流程一致。

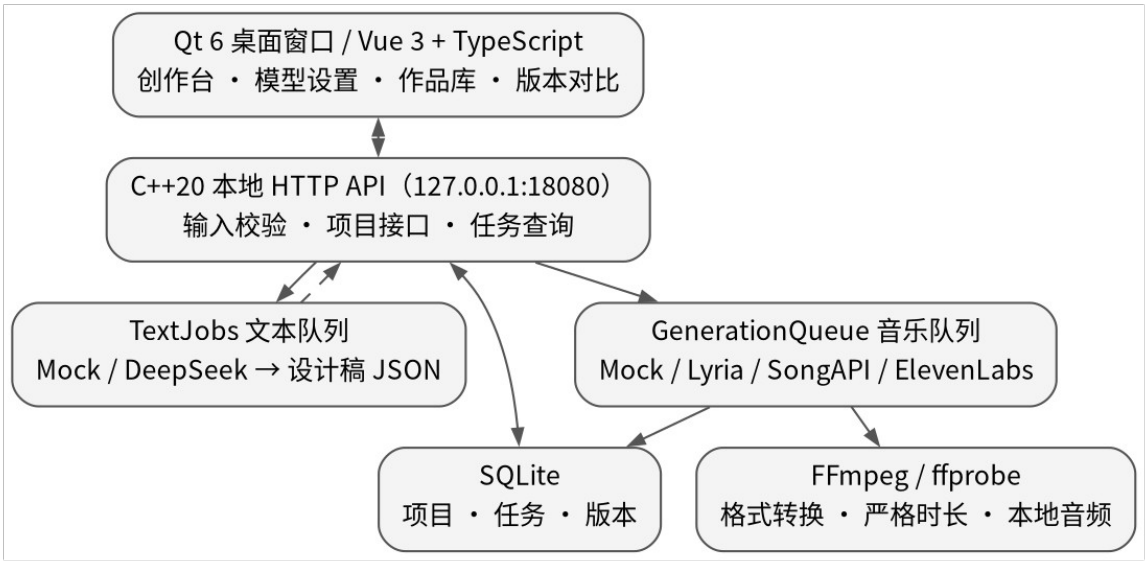


图 2-1 系统架构与模块关系

模块及主要文件	职责与设计理由
models/blueprint.cpp、schema.cpp	MusicBlueprint 为统一数据结构；Schema 验证类型、范围和必填项，再检查段落重叠等业务规则。
providers/、services/text_jobs.cpp	文本和音乐使用独立接口；文本任务单独排队，修改设计稿不占用音乐任务队列。
services/generation_queue.cpp	保存配置快照后由工作线程执行；依次更新准备、生成和后处理状态，避免 HTTP 线程等待长任务。
repositories/database.cpp	SQLite 保存项目、任务与版本；音频作为独立文件存放，以 UUID 关联并避免覆盖。
frontend/src/、desktop/src/	Pinia 管理页面状态，Vue Router 切换页面；Qt 管理单实例、下载对话框、退出确认和后端进程。

2.2 数据关系与异常边界

projects 保存项目及当前草稿，tasks 保存生成状态和请求快照，versions 保存完成结果与评价。关系为“一个项目—多个任务—多个版本”；parentVersionId 表示版本父节点，draftParentVersionId 表示正在编辑的分支来源。外键、同项目检查和锁内操作共同约束删除与生成竞态。运行中断的任务会标记失败避免重启后被误当作继续计费的任务。

3 系统实现

3.1 提示词模板与结构化设计稿

DeepSeekProvider::design 将角色、输出约束、JSON Schema 和样例组织为系统提示词，再把场景或修改要求作为用户消息。模型仅负责生成设计稿，不允许自由执行工具。输出需经过 JSON 解析、Schema 校验和业务校验，格式不合格最多修正一次；鉴权失败、网络超时和限流不会按格式错误重复提交。

```
     // backend/src/providers/deepseek.cpp（节选）
json messages = json::array({
    {{"role", "system"}, {"content", system}},
    {{"role", "user"}, {"content", request}}
});
for (int attempt = 0; attempt < 2; ++attempt) {
    auto content = completion(messages, 8192);
    try {
        const auto value = parseProviderJson(content);
        return parseModelBlueprint(value);
    } catch (const json::exception&) { issue = "输出不是完整 JSON 对象或字段类型无效"; }
    catch (const ValidationError& error) { issue = error.what(); }
    // 原函数随后追加脱敏后的校验反馈；完整敏感信息检查见附件源码。
}
```

本地 composePrompt 同时生成场景、BPM、乐器、歌词和“[开始秒–结束秒]”段落描述。设计稿保存 promptFingerprint；参数变化时重新生成提示词，避免出现界面参数已更新而实际请求仍使用旧提示词的情况。finalLyriaPrompt 是历史字段名，目前被多个音乐适配器共用。

3.2 受约束 Agent 工作流与任务调度

应用把“生成设计稿、校验、音乐合成、音频处理、持久化”组合为有限步骤的创作助手。模型提供规划数据，用户确认可见参数后，C++ 调度器才执行音乐工具；模型没有自主 function calling 或无限循环权限。这种应用级编排强调人机协作与可解释状态，避免把一次自然语言输入直接变成不可控的外部调用。

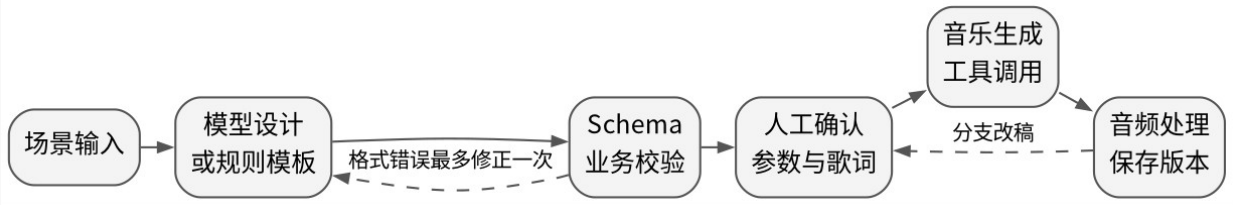


图 3-1 人工确认与固定工具链构成的创作工作流

```
     工作流伪代码（对应 studio.ts 与 generation_queue.cpp）：
接收场景 → 请求设计稿 → Schema/业务校验 → 展示可编辑参数
用户确认生成：保存项目，冻结设计稿、模型配置与父版本 ID
任务入队：queued → preparing → generating → post_processing
成功：保存原音频与成品，事务写入版本 → completed
失败：记录可读错误 → failed；不自动重发收费生成请求
用户选择旧版本：复制参数为分支草稿，再进入确认步骤
```

3.3 ElevenLabs 音乐模型接入

新增 ElevenLabsProvider 实现 IMusicModelProvider。设置页提供服务商选择、独立密码输入框和连接测试；后端通过 xi-api-key 请求头认证，地址固定为官方域名。连接测试调用 /v1/music/plan，只验证编排接口；实际音乐调用 /v1/music。两种测试分开，避免把缺少 user_read 权限误判为音乐 Key 无效。

```
    // backend/src/providers/elevenlabs.cpp（节选）
const double seconds = request.mode == "preview"
    ? 30.0 : request.blueprint.targetDurationSeconds;
const json body{
    {"prompt", prompt},
    {"music_length_ms", std::llround(seconds * 1000)},
    {"model_id", "music_v1"},
    {"force_instrumental", request.blueprint.vocalMode == "instrumental"}
};
const auto bytes = requestElevenLabs(
    "/v1/music?output_format=mp3_44100_128", settings_.apiKey,
    body, {10, 600, cancelled_});
// 返回值检查 MP3 特征后写入独立文件，交给统一后处理流程。
```

提示词按 Unicode 字符计数，上限 4100 字符，超限报错而不截断歌词；纯音乐通过 force_instrumental 强制约束。完整歌曲按目标秒数转换为毫秒，试听按 30 秒缩放时间轴。官方同步返回的音频由后台工作线程等待，因此本机 HTTP 请求仍能及时返回任务 ID。生成结果统一保存为可查询任务和作品版本。

Key 分服务商保存在后端会话内存，公开设置只返回 hasApiKey、keySource 和状态。非敏感偏好才写入 settings.json；重启后可重新填写 Key 或设置 ELEVENLABS_API_KEY。HTTP 层限制响应大小、启用 TLS 校验并禁止重定向，对付费套餐、权限、余额、限流等错误提供固定中文说明，不回显远端原始错误文本。

3.4 音频后处理与严格时长

生成音频先保存为 .source.mp3/.source.wav，再转换成用户选择的成品格式。严格时长始终输出 44.1 kHz、双声道、16-bit PCM WAV：短音频补静音，长音频淡出并裁剪，最后由 ffprobe 校验实际时长。补静音不等于续写旋律，裁剪也不代表模型准确遵守了每个音乐段落。

```
    // backend/src/services/audio_processing.cpp（节选）
const double fadeEnd = std::min(original, target);
const double fade = std::min(2.0, fadeEnd / 2);
// FFmpeg 参数：afade=t=out:st=...:d=...,apad,atrim=duration=...
auto actual = probe(destination, outputFormat, cancelled);
if (strict && std::abs(actual - target) > 1.0 / 44100 + 0.000001)
    throw ProviderError("duration_mismatch", "处理后音频未满足严格时长，作品未保存");
return {original, actual};
```

本地处理失败时保留已取得的原始音频供恢复，不把失败任务展示为作品；界面分别显示模型目标时长、源音频时长与成品时长。

3.5 持久化、版本分支与对比

Database::completeTask 在一个事务中写入作品并更新任务状态，避免出现“任务已完成但没有作品”的部分提交。每次生成使用新的 UUID，既保留设计参数与实际提示词，也保存模型名称、格式、时长和父版本关系。评分和备注单独修改，音频参数快照保持不可变。

```
    // backend/src/repositories/database.cpp（节选）
execute(db_, "BEGIN IMMEDIATE");
try {
    Statement query(db_, "INSERT INTO versions(id,project_id,task_id,payload) "
                         "VALUES(?,?,?,?)");
    query.bind(1, version.at("id")); query.bind(2, task.at("projectId"));
    query.bind(3, id); query.bind(4, version.dump()); query.next();
    writeTask(db_, task); execute(db_, "COMMIT");
} catch (...) { execute(db_, "ROLLBACK"); throw; }
```

从历史版本创建分支时只复制设计稿；再次生成后才形成子版本。父版本有子节点或活跃生成任务时拒绝删除，减少悬空引用。A/B 对比同时载入两个音频，切换时保留播放位置；只有当前侧播放，短音频将位置限制到有效范围。

3.6 前后端框架应用与桌面部署

Vue 组件负责表单、时间轴、波形和路由，Pinia 负责工作流状态。本地 POST /api/generations 返回 HTTP 202；前端随后轮询任务，依据状态更新进度和播放器。使用 pollingEpoch 使旧轮询失效，避免切换项目后旧结果覆盖新页面。设置页等待初始配置加载完成后才允许编辑，防止迟到响应覆盖用户刚选择的服务商。

```
    // frontend/src/stores/studio.ts（节选）
const result = await api<GenerationTask>(`/generations/${id}`)
if (epoch !== pollingEpoch) return
 task.value = result; error.value = ''
if (result.status === 'failed') error.value = result.error
else if (result.status !== 'completed')
    timer = setTimeout(() => { void poll(id, epoch) }, 400)
```

开发时 Vite 将 /api 代理到 C++；生产构建后由后端托管 frontend/dist。Qt WebEngine 使用内存会话承载页面，本地请求拦截器限制外部导航；桌面外壳启动并核验自己的后端实例，接管文件下载和退出确认。重复启动激活已有窗口，关闭时清理本机后台进程。

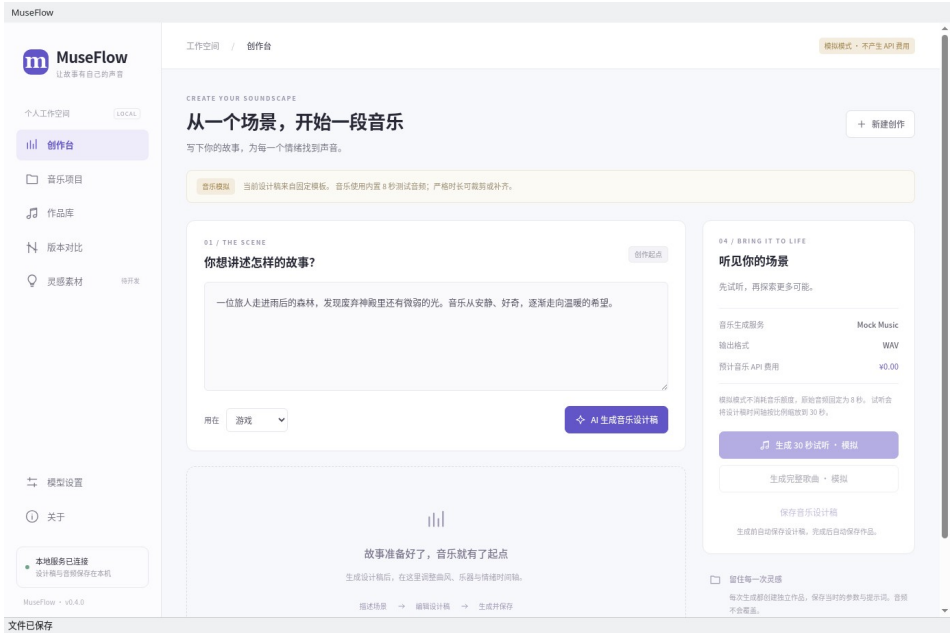


图 3-2 桌面运行截图：场景输入与生成入口（原生窗口测试）

3.7 典型产品项目的全过程实现

以“雨后森林中的旅人”场景为例：输入从安静、好奇走向温暖希望的配乐需求，选择游戏用途；创作助手生成含曲风、BPM、乐器与时间轴的设计稿。用户检查段落起止时间，按需调整歌词或改为纯音乐，再保存为一个独立项目。

在模型设置选择 ElevenLabs Music，填入具有音乐权限的 Key，先测试连接，再生成 30 秒试听。试听阶段将完整设计稿时间轴等比例缩放，完整设计参数仍保留在快照中。生成后可播放和下载；若需精确配合画面，开启严格时长。随后从已完成版本创建分支，改变强度或 BPM，生成候选版本并进行 A/B 比较，最后给出评分、备注，导出项目记录与选定音频。

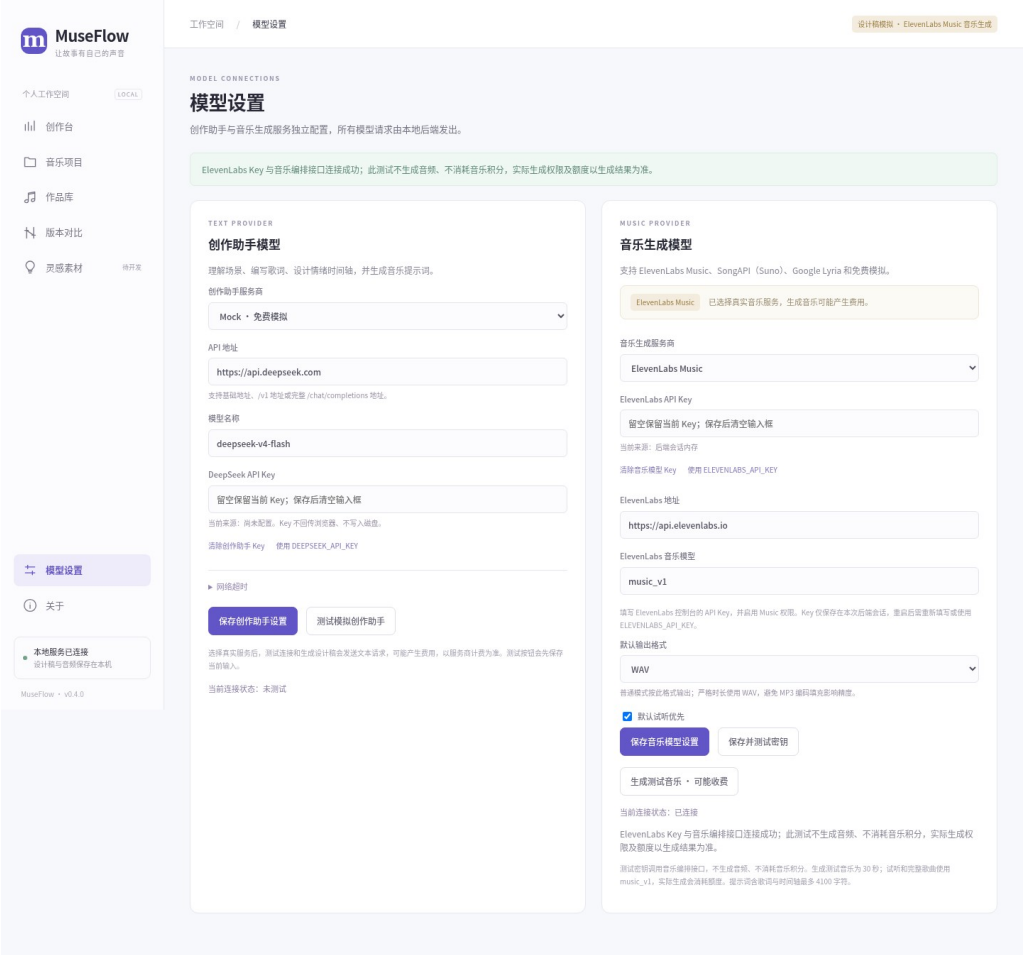


图 3-3 ElevenLabs 设置与连接成功状态（本地契约测试，使用假 Key）

上述过程覆盖需求输入、模型规划、人工编辑、工具执行、后处理、保存、比较和交付。截图用于说明真实程序界面；契约测试截图不作为官方音乐质量或生成时长的证明，真实服务调用结果另见第 4 章。

4 系统测试

4.1 测试方法与覆盖范围

测试分为核心单元、HTTP 集成、服务商契约、浏览器和 Qt 桌面五层。自动化测试使用临时 XDG 目录和本机假服务，不改动日常项目，也不调用收费音乐模型。真实账户测试只验证已明确执行的 ElevenLabs 路径，不能由本地契约测试推断其他服务商在线可用。

测试项	输入/操作与预期	验证结果
C++ 核心	字段与时间轴校验、任务和数据库约束、配置持久化	CTest 核心测试通过
前端单元	参数校验、版本差异计算、API 客户端错误处理	Vitest 3 个测试文件、16 个用例通过
HTTP 集成	完整生成流程、健康检查、Range 下载、重启恢复、跨来源限制	通过；长任务期间本机接口可响应
DeepSeek / Lyria 契约	本地模拟远端响应，覆盖无效 JSON、Base64、权限、限流及超时	通过；仅格式错误有一次有限修正
ElevenLabs 契约	独立 Key、编排测试、毫秒时长、人声参数、MP3/WAV、错误分类	通过；超长提示词不提交；失败不自动重发
浏览器端到端	设置与连接、生成与下载、编辑、严格时长、分支、评分与导出	Playwright 6 个场景通过
Qt 桌面	单实例、环境变量传递、创建与播放、下载、退出及重启	桌面回归通过；原生 JSON 保存检查首次失败，单独复测通过

4.2 ElevenLabs 真实调用结果

2026 年 9 月 10 日，先直接提交 3 秒 music_v1 纯音乐请求，接口返回 HTTP 200、audio/mpeg，得到 48,946 字节 MP3；ffprobe 检测时长为 3.056313 秒，说明编码容器时长与目标存在差异。随后通过编译后的 MuseFlow C++ 后端执行音乐编排连接测试，任务成功完成。

再通过本机 /api/generations 提交 5 秒严格时长生成任务，完成真实音乐调用、音频处理和版本保存。源音频时长为 5.041625 秒，最终 WAV 为 5.000000 秒、882,078 字节，provider 为 elevenlabs、model 为 music_v1。该结果验证了鉴权、后台调用、解析、格式转换、严格时长和下载的完整链路。

4.3 问题修正与结果边界

接口联调中，免费套餐返回 paid_plan_required，升级后才可生成；用户信息接口缺少 user_read 权限不影响音乐生成，因此连接测试改用音乐编排接口。浏览器回归发现初始设置请求可能覆盖用户选择，修复为加载完成后启用输入控件。共享 HTTP、存储与前端流程的原有回归在修改后重新验证通过。

当前测试确认功能链路和错误处理，不评价旋律审美、长期稳定性或大规模并发。真实在线验证仅覆盖 ElevenLabs；DeepSeek、Lyria 和 SongAPI 的在线账户可用性仍由具体 Key、权限、地区及服务状态决定。

5 项目安装配置文档

安装 Linux C++/Qt 开发依赖、Node.js 和 FFmpeg 后，将源码附件解压为 museflow 目录。先检查 cmake、g++、node、npm、ffmpeg 和 ffprobe 均可执行，再在项目根目录运行：

```
./scripts/build.sh              # 构建 C++ 后端、Qt 桌面及 Vue 静态资源
./scripts/run.sh                # 启动独立桌面窗口
./scripts/install-desktop.sh    # 可选：注册当前用户的应用菜单入口
./scripts/test.sh --e2e         # 可选：单元、集成及浏览器回归
```

首次构建会安装 npm 锁定依赖，需要网络。程序默认 Mock；真实使用时进入“模型设置”，选择文本和音乐服务商，填写 Key 后保存并测试。ElevenLabs 使用 music_v1；音乐 Key 留空表示保留现有 Key，清除和恢复环境变量有独立按钮。程序不会自动加载 .env，附件中的 .env.example 只用于说明。

默认数据目录为 ~/.local/share/museflow，设置目录为 ~/.config/museflow，缓存目录为 ~/.cache/museflow（可由 XDG 环境变量覆盖）。只监听本机 18080 端口；如果被占用，应先关闭对应开发服务。构建/启动参数、发行版依赖包及故障排查详见附件 README.md。

6 项目总结

本项目完成了一个可运行的本地 AI 配乐产品：将场景描述转成可编辑设计稿，通过可替换适配器生成音频，再提供严格时长、持久化与版本对比。主要收获是把不确定的模型输出放入明确的软件边界：生成前校验、执行时隔离队列、结果保存事务化、历史版本不覆盖、错误对用户可解释。

实现中需要区分模型能力与应用能力：提示词只能引导结构，不能保证音乐段落完全按秒执行；严格时长由本机后处理完成；改稿会生成新版本，并非直接编辑旧音轨。后续可增加 Linux Secret Service、AppImage 分发和更完整的音乐结构控制。素材管理、向量检索、RAG 和自主工具选择目前未实现，需另行评估需求与成本。

附录：全部源代码电子附件

完整源码随本报告提交，共 100 个项目文件，包含业务代码、前端页面、桌面外壳、测试、构建配置、Schema、资源和第三方源码许可证。为减少打印页数，正文仅摘录关键代码，全部内容通过下列电子附件提供。

查看全部源代码（离线 HTML，支持按文件名筛选）

下载完整源码 ZIP（解压后可构建运行）

请将报告与上述两个附件保存在同一目录。Word/WPS 中可按住 Ctrl 点击链接；阅读附件无需联网。如阅读器未直接打开链接，也可按上述文件名在同目录打开。

附件目录	内容
backend/、desktop/	C++ 模型接口、队列、数据库、音频处理、HTTP 服务与 Qt 窗口
frontend/	Vue/TypeScript/CSS、页面与组件、状态管理、依赖锁文件、浏览器测试
scripts/、assets/	构建运行和回归脚本、契约测试服务、Mock 音频及资源说明
README.md、.env.example	安装使用说明和空白环境变量示例；不包含真实 Key 或个人项目数据